LANGUAGES AND AUTOMATA I

HOMEWORK #3 PARTIAL ANSWERS

exercises.ml

```
exception Negative_Amount

let change (amount: int) =
  if amount < 0 then
    raise Negative_Amount
  else
    let denominations = [25; 10; 5; 1] in
    let rec aux remaining denominations =
      match denominations with
      | [] -> []
      | d :: ds -> (remaining / d) :: aux (remaining mod d) ds
    in
    aux amount denominations

let first_then_apply (array: 'a list) (predicate: 'a -> bool) (consumer: 'a -> 'b
  match List.find_opt predicate array with
  | Some x -> consumer x
  | None -> None

(*
  This also works:
  List.find_opt predicate array |> Option.map consumer |> Option.join
*)

let powers_generator base =
  let rec gen_from power () =
    Seq.Cons (power, gen_from (power * base))
  in
  gen_from 1

let meaningful_line_count filename =

  let is_meaningful_line line =
```

```
    let trimmed = String.trim line in
    trimmed <> "" && not (String.starts_with ~prefix:"#" trimmed)
  in

  let ic = open_in filename in
  let finally () = close_in ic in

  let rec count_lines count =
    match input_line ic with
    | line ->
      let new_count = if is_meaningful_line line then count + 1 else count in
      count_lines new_count
    | exception End_of_file ->
      count
  in
  Fun.protect ~finally (fun () -> count_lines 0)

type shape =
  | Sphere of float
  | Box of float * float * float

let volume s =
  match s with
  | Sphere r -> (4.0 /. 3.0) *. Float.pi *. (r ** 3.0)
  | Box (w, l, d) -> w *. l *. d

let surface_area s =
  match s with
  | Sphere r -> 4.0 *. Float.pi *. (r ** 2.0)
  | Box (w, l, d) -> 2.0 *. ((w *. l) +. (w *. d) +. (l *. d))

type 'a binary_search_tree =
  | Empty
  | Node of 'a * 'a binary_search_tree * 'a binary_search_tree

let rec size tree =
  match tree with
  | Empty -> 0
  | Node (_, left, right) -> size left + size right + 1

let rec insert data tree =
  match tree with
  | Empty -> Node (data, Empty, Empty)
  | Node (v, left, right) ->
    if data < v then
      Node (v, insert data left, right)
```

```
    else if data > v then
      Node (v, left, insert data right)
    else
      tree

let rec contains data tree =
  match tree with
  | Empty -> false
  | Node (v, left, right) ->
    if data < v then
      contains data left
    else if data > v then
      contains data right
    else
      true

let rec inorder tree =
  match tree with
  | Empty -> []
  | Node (v, left, right) -> inorder left @ [v] @ inorder right
```

Exercises.hs

```
module Exercises
    ( change,
      firstThenApply,
      powers,
      meaningfulLineCount,
      Shape(..),
      BST(Empty),
      volume,
      surfaceArea,
      size,
      contains,
      insert,
      inorder
    ) where

import qualified Data.Map as Map
import Data.List(isPrefixOf, find)
import Data.Char(isSpace)

change :: Integer -> Either String (Map.Map Integer Integer)
change amount
    | amount < 0 = Left "amount cannot be negative"
    | otherwise = Right $ changeHelper [25, 10, 5, 1] amount Map.empty
```

```
        where
          changeHelper [] remaining counts = counts
          changeHelper (d:ds) remaining counts =
            changeHelper ds newRemaining newCounts
              where
                (count, newRemaining) = remaining `divMod` d
                newCounts = Map.insert d count counts

firstThenApply :: [a] -> (a -> Bool) -> (a -> b) -> Maybe b
firstThenApply xs p f = f <$> find p xs

powers :: Integral a => a -> [a]
powers base = map (base^) [0..]

meaningfulLineCount :: FilePath -> IO Int
meaningfulLineCount filePath = do
    document <- readFile filePath
    let allWiteSpace = all isSpace
        trimStart = dropWhile isSpace
        isMeaningful line =
            not (allWiteSpace line) &&
            not ("#" `isPrefixOf` (trimStart line))
    return $ length $ filter isMeaningful $ lines document

data Shape
    = Sphere Double
    | Box Double Double Double
    deriving (Eq, Show)

volume :: Shape -> Double
volume (Sphere r) = (4 / 3) * pi * (r ^ 3)
volume (Box w l d) = w * l * d

surfaceArea :: Shape -> Double
surfaceArea (Sphere r) = 4 * pi * (r ^ 2)
surfaceArea (Box w l d) = 2 * ((w * l) + (w * d) + (l * d))

data BST a = Empty | Node a (BST a) (BST a)

insert :: Ord a => a -> BST a -> BST a
insert x Empty = Node x Empty Empty
insert x (Node y left right)
    | x < y = Node y (insert x left) right
    | x > y = Node y left (insert x right)
    | otherwise = Node y left right
```

```
contains :: Ord a => a -> BST a -> Bool
contains _ Empty = False
contains x (Node y left right)
    | x < y = contains x left
    | x > y = contains x right
    | otherwise = True

size :: BST a -> Int
size Empty = 0
size (Node _ left right) = 1 + size left + size right

inorder :: BST a -> [a]
inorder Empty = []
inorder (Node x left right) = inorder left ++ [x] ++ inorder right

instance (Show a) => Show (BST a) where
    show :: Show a => BST a -> String
    show Empty = "()"
    show (Node x Empty Empty) = "(" ++ show x ++ ")"
    show (Node x Empty right) = "(" ++ show x ++ show right ++ ")"
    show (Node x left Empty) = "(" ++ show left ++ show x ++ ")"
    show (Node x left right) = "(" ++ show left ++ show x ++ show right ++ ")"
```

exercises.ts

```
import { open } from "node:fs/promises"

export function change(amount: bigint): Map<bigint, bigint> {
  if (amount < 0) {
    throw new RangeError("Amount cannot be negative")
  }
  let counts: Map<bigint, bigint> = new Map()
  let remaining = amount
  for (const denomination of [25n, 10n, 5n, 1n]) {
    counts.set(denomination, remaining / denomination)
    remaining %= denomination
  }
  return counts
}

export function firstThenApply<T, U>(
  array: T[],
  predicate: (x: T) => boolean,
  consumer: (x: T) => U | undefined
): U | undefined {
  const first = array.find(predicate)
```

```
  if (first) {
    return consumer(first)
  }
  return undefined
}

export function* powersGenerator(base: bigint): Generator<bigint> {
  for (let power = 1n; true; power *= base) {
    yield power
  }
}

export async function meaningfulLineCount(filename: string): Promise<number> {
  let count = 0
  const file = await open(filename, "r")
  for await (const line of file.readLines()) {
    const trimmed = line.trim()
    if (trimmed && !trimmed.startsWith("#")) {
      count++
    }
  }
  return count
}

interface Sphere {
  kind: "Sphere"
  readonly radius: number
}

interface Box {
  kind: "Box"
  readonly width: number
  readonly length: number
  readonly depth: number
}

export type Shape = Sphere | Box

export function volume(shape: Shape): number {
  switch (shape.kind) {
    case "Sphere":
      return (4 / 3) * Math.PI * shape.radius ** 3
    case "Box":
      return shape.width * shape.length * shape.depth
  }
}
```

```
export function surfaceArea(shape: Shape): number {
  switch (shape.kind) {
    case "Sphere":
      return 4 * Math.PI * shape.radius ** 2
    case "Box":
      const { length, width, depth } = shape
      return 2 * (length * width + length * depth + width * depth)
  }
}

export interface BinarySearchTree<T> {
  size(): number
  insert(data: T): BinarySearchTree<T>
  contains(data: T): boolean
  inorder(): Iterable<T>
}

export class Empty<T> implements BinarySearchTree<T> {
  size(): number {
    return 0
  }
  insert(data: T): BinarySearchTree<T> {
    return new Node<T>(data, new Empty<T>(), new Empty<T>())
  }
  contains(data: T): boolean {
    return false
  }
  *inorder(): Iterable<T> {}
  toString(): string {
    return "()"
  }
}

class Node<T> implements BinarySearchTree<T> {
  constructor(
    private readonly data: T,
    private readonly left: BinarySearchTree<T>,
    private readonly right: BinarySearchTree<T>
  ) {}

  size(): number {
    return this.left.size() + this.right.size() + 1
  }

  insert(data: T): BinarySearchTree<T> {
```

```
    if (data < this.data) {
      return new Node(this.data, this.left.insert(data), this.right)
    } else if (data > this.data) {
      return new Node(this.data, this.left, this.right.insert(data))
    } else {
      return this
    }
  }

  contains(data: T): boolean {
    if (data < this.data) {
      return this.left.contains(data)
    } else if (data > this.data) {
      return this.right.contains(data)
    } else {
      return true
    }
  }

  *inorder(): Iterable<T> {
    yield* this.left.inorder()
    yield this.data
    yield* this.right.inorder()
  }

  toString(): string {
    const left = this.left instanceof Empty ? "" : this.left.toString()
    const right = this.right instanceof Empty ? "" : this.right.toString()
    return `(${left}${this.data}${right})`
  }
}
```